



OCP

OpenCognition Protocol

"Where Minds Meet Machines"

OCP Complete Git Repository Overview

Here's the complete Git technical specification for OCP. It's designed to be the single source of truth for anyone contributing to or maintaining the repository. A few notes on what it covers and why:

Repository structure (§3) maps every file in the repo to an owning team with explicit approval requirements via CODEOWNERS. This prevents spec changes from sneaking through SDK reviews and vice versa.

Branching model (§4) uses trunk-based development with short-lived feature branches because OCP has multiple independent artifacts (spec, schemas, SDK, node) that need to move at different speeds. Release branches are cut per-artifact, not globally, which is why tags use the `<artifact>/<version>` format rather than a single version number.

Commit convention (§5) is Conventional Commits with OCP-specific scopes that match directory names. This enables automated changelog generation and version bumping — a `feat(spec)!` commit automatically signals a major protocol version bump.

Versioning (§6) deliberately separates protocol version (`MAJOR.MINOR`) from software versions (`MAJOR.MINOR.PATCH`). This is critical because the spec and SDK evolve at different rates — a bug fix in the SDK shouldn't imply a protocol change.

Release process (§7) has a 13-step checklist with different approval thresholds per release type. Major releases require TSC supermajority and 12 months deprecation notice, matching the governance model from the spec.

CI/CD (§8) defines five workflows with concrete YAML. The pipeline gates are strict — every PR must pass lint, typecheck, tests (across three Python versions), compliance, schema validation, Docker build, and integration tests.

Contribution workflow (§9) has three tracks: standard (features/fixes), RFC (spec changes), and extension proposals. The RFC process adds a 14-day comment period and higher approval requirements, reflecting that spec changes affect the entire ecosystem.

Quality standards (§10) are prescriptive about everything from schema conventions (additionalProperties: false, description on every property) to Python style (structlog not print, Google docstrings, no assert for validation).

OCP Complete Python SDK Overview

Here's the complete Python SDK technical specification — the document an implementer would need to build a conforming OCP Python SDK from scratch, or to verify that the reference implementation is correct.

A few things worth highlighting about the design decisions encoded in this spec:

API contracts are behavioral, not just structural. Every method has explicit pre-conditions, post-conditions, and error semantics. For example, `MessageBuilder.build()` doesn't just "build a message" — the spec defines the exact signing algorithm (Ed25519 over SHA-3-256 of RFC 8785 canonical JSON with the signature field zeroed), the exact message ID format, and the exact exception type raised on each failure mode.

The PVL check order is normative (§10.2). Size is checked first because it's cheapest, then provenance, then anonymization, then differential privacy, and finally PII scanning (most expensive). The "first failure wins" rule means a payload that's both oversized and contains PII gets rejected with PVL-005, not PVL-001. This matters for error reporting consistency across implementations.

Thread safety is explicitly documented per-class (§16.2). Immutable types (`SigningKeyPair`, `EncryptionKeyPair`, crypto functions, PVL functions) are thread-safe. Mutable types (`Agent`, `ConsensusRound`, `AgentIdentity`) are not. This prevents a class of concurrency bugs that would otherwise surface in production without clear guidance.

The dependency policy (§18.3) is deliberately restrictive. No ML frameworks, no database libraries, no web frameworks. The SDK is a protocol library — it should be embeddable in any Python application without pulling in a heavy dependency tree. The sole exception (GF(2⁸) arithmetic) is explicitly called out and justified.

Conformance levels (§19.1) match the protocol's layered design. An implementation that only needs messaging (Layers 1–2) can target "OCP Core SDK" without implementing knowledge exchange or consensus. This lowers the barrier for partial adoption.

Backward compatibility rules (§19.3) are concrete — no public symbol removal, no required parameter additions, no return type changes within a major version. This gives downstream users a stable target.

OCP Security Overview

Here's the complete security technical specification. This is the document a security auditor, penetration tester, or implementation team would use to evaluate or build a conforming OCP deployment. A few notes on the key decisions:

The "No Backdoors" principle (§2.4) is explicitly non-negotiable. It's the only principle described as "not subject to governance votes." This is deliberate — the most common way backdoors enter protocols is through governance pressure, and the spec forecloses that path at the constitutional level.

Cryptographic requirements (§3.2) are unusually prescriptive. Most protocol specs say "use Ed25519" and leave it there. This spec goes deeper — rejecting non-standard variants (Ed25519ph, Ed25519ctx), requiring full curve-point validation, mandating clamping for X25519, prohibiting GCM tag truncation, and explicitly banning a dozen algorithms with reasons. This matters because implementation choices within "use Ed25519" can create real vulnerabilities.

The PVL non-bypassability clause (§8.1.2) is one of the strongest statements in the spec. There is no "disable PVL" flag, no "force send" override, no "warn-only" mode. Removing the PVL call produces a non-conforming implementation by definition. This is the protocol's answer to the question "what if both parties consent to sharing PII?" — the answer is "the protocol doesn't allow it."

The post-quantum section (§3.4) goes beyond the typical "we'll deal with it later" hand-wave. It defines specific algorithm identifiers, a concrete hybrid migration strategy (dual KEM, dual signatures, composite DID keys), a five-phase roadmap with dates, and a threat assessment that distinguishes harvest-now-decrypt-later (HIGH urgency) from signature forgery (MEDIUM urgency — requires fault-tolerant quantum computers that don't exist yet).

The incident response section (§13) includes something most protocol specs omit: specific procedures for agent compromise and custodian compromise. These are the scenarios that actually happen in production, and having a pre-defined response procedure prevents ad-hoc decisions under pressure.

The compliance checklist (§15.1) is designed to be mechanically verifiable — 20 specific checks, each with a defined verification method. An auditor can go down the list and produce a pass/fail for each item without subjective judgment.

OCF Complete Docker Overview

the complete Docker technical specification. It covers everything an infrastructure team needs to build, deploy, operate, and secure OCF containers. Some notes on the key decisions:

Single image, multiple components (§2.2). All three services share one Dockerfile and are differentiated at runtime by `--component`. This keeps the build matrix simple (one image to scan, sign, and distribute) while allowing services to run as isolated containers in production.

Layer ordering for cache optimization (§2.2). The Dockerfile copies `pyproject.toml` and `requirements.txt` before source code. This means dependency layers are cached — a code change doesn't trigger a full pip install, cutting build times from minutes to seconds during development.

Non-root, read-only, zero capabilities (§8.1). The container runs as a dedicated `ocf` user, with a read-only root filesystem (`tmpfs` for `/tmp`), all Linux capabilities dropped, and `no-new-privileges` set. This is the maximum hardening you can apply without breaking functionality — even if an attacker gets code execution inside the container, they can't escalate or persist.

Custodian isolation (§4.3, §8.4). The custodian holds encrypted recovery shares — the most sensitive data in the system. The spec recommends it NOT be exposed to the public internet and only be accessible through the transport node or an authenticated reverse proxy. The custodian volume MUST be encrypted at rest in production.

Three deployment topologies (§10). Development (single-host Compose), single-org production (load-balanced transport nodes, federated registry, isolated custodian), and federated multi-org (each org runs a full stack, registries gossip-sync). The Kubernetes manifests in §10.4 include the security context matching the Docker hardening.

Storage is SQLite by design (§5.2). This is a deliberate choice for the reference implementation — no external database dependency, simple backup (copy one file), and sufficient performance for the reference scale. Production deployments at scale would replace SQLite with PostgreSQL, but that's an operator decision, not a protocol requirement.

Performance benchmarks (§12.1) are on reference hardware (4 vCPU, 8 GB RAM) so operators can extrapolate to their own infrastructure. The targets are conservative — 1,000 HTTP msg/s, 2,000 WS msg/s, 1,000 concurrent connections.